



Atualização da pasta: EXT_Commercial_Data

Plataforma: Qlik.

Caminho: Services ETL → EXT_Commercial_Data

- Esta aplicação extrai dados das tabelas **FatoCompra** e **FatoVenda** e realiza correções conforme necessário.
- **Funcionalidades:**
 -  **Correção retroativa (mês):** O mês é processado por completo, dividido em períodos de 7 dias, começando do último dia até o primeiro. Se houver divergências em algum período, o script percorre os dias desse intervalo para identificar exatamente em quais dias as inconsistências ocorrem e corrigi-las individualmente.

Módulos (arquivos TXT):

Separamos alguns scripts em módulos para facilitar a manutenção do código. Esses módulos incluem:

- Variáveis de configuração da aplicação principal (Main).
- Caminhos ("path") de conexões.
- Sub-rotinas responsáveis pela geração de arquivos QVD.
- Sub-rotinas utilizadas para a extração de dados

Módulos são arquivos que contêm código reutilizável e podem ser importados por diferentes aplicações

```
/*  
    Esse código busca as variáveis de configuração Main (Variáveis de controle de datas e conversões)  
    Também possui a variável de controle de duração da carga (vStartLoad)  
*/  
$(Include=[lib://Services ETL:DataFiles/VAR_MAIN_PTBR.txt])  
  
/*  
    Esse código busca as variáveis de conexão  
    Variáveis:  
    caminho da extração: vExtractedPath = 'Lib://Services ETL:DataFiles';  
    conexão com o Banco de Dados: vConectionC5 = 'Desenvolvimento:Consinco';  
    conexão: LIB CONNECT TO '$(vConectionC5)';  
*/  
$(Include=[lib://Services ETL:DataFiles/VAR_PATH_CONNECTION_C5.txt])  
  
/*  
    Esse código busca as SubRotinas de criação de arquivos e Registros de tempos de carga  
    SubRotina de criação QVD:  
    RecordQVD(pTable) > recebe como parâmetro o nome da tabela para criação do arquivo  
  
    SubRotina de criação CSV:  
    RecordCSV(pTable,vFile) > recebe como parâmetro o nome da tabela (pTable) que será criado o QVD,  
    (vFile) recebe uma nomenclatura que o nome do arquivo QVD irá receber  
  
    SubRotina para registrar o tempo de leitura das extrações  
    Essa SubRotina é chamada pelas SubRotinas de criação de arquivos (RecordQVD, RecordCSV)  
    ReadingTime(pTable,pInitTime,pFinalTime) > Recebe como parametro o nome da tabela que foi gerada (pTable)  
    O tempo que começou a carga e o tempo que finalizou (pInitTime,pFinalTime)  
  
    SubRotina para armazenar os registros de leitura e extrações em uma tabela que fica na memória do qlick  
    Essa SubRotina é chamada pela SubRotina(ReadingTime).  
    ReadingTime(pTable,pInitTime,pFinalTime) > Recebe como parametro o nome da tabela que foi gerada (pTable)  
    e o tempo de duração da carga (pInitTime,pFinalTime)  
*/  
$(Include=[lib://Services ETL:DataFiles/SUB_RECORDQVD.txt])
```

Seção Carga completa: Nela teremos a estrutura principal da nossa carga, como por exemplo:

- 1) Variáveis de configuração para regência de data, onde a variável principal é a vYesterday.
- 2) Verifica o dia para gerar a carga de correção do mês anterior, se o dia for menor ou igual a 7 a carga será lançada desde o mês passado.
- 3) Sub-rotina RecoverMonths (recuperar os meses). Ela é a estrutura principal da nossa carga, é através dela que corrigimos um ou mais meses. Ela recebe 4 parâmetros essenciais para as suas funcionalidades, todos de caráter obrigatórios.
- 4) O encerramento do script, captura a data e a hora do encerramento do script, executa uma sub-rotina para registrar o tempo total da carga e sai do script.

```
LET vYesterday = Today()-1;  
LET vDay = Day(vYesterday);  
LET vInitMonth = 0;
```

1

```
// Verifica Se o dia é menor ou igual a 7 e aplica a carga retroativa mensal, acumulativa até dia - 1  
If vDay <= 7 then  
    vInitMonth = -1;  
end if;
```

2

```
/*  
Legenda dos Parâmetros:  
1º Mês inicial (sempre preencher com valores negativos).  
2º mês final, qual o mês que a carga deve parar, sendo 0 o mês atual;  
3º Qual tabela carregará, 1 = venda, 2 = Compra, 3 = ambas.  
4º Recebe o dia de ontem  
*/
```

```
CALL RecoverMonths(vInitMonth,0,'3',vYesterday);  
// CALL FATO_VENDA_VALID;
```

3

```
LET vFullload = NOW();  
CALL ReadingTime('TOTAL',vStartLoad,vFullload);  
Exit Script;
```

4

```
/*
```

Legenda dos Parâmetros:

1º Mês inicial (sempre preencher com valores negativos).

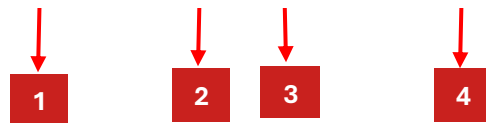
2º mês final, qual o mês que a carga deve parar, sendo 0 o mês atual;

3º Qual tabela carregará, 1 = venda, 2 = Compra, 3 = ambas.

4º Recebe o dia de ontem

```
*/
```

```
CALL RecoverMonths(vInitMonth,0,'3',vYesterday);
```



Seção Carga completa: Nela teremos a estrutura principal da nossa carga a sub-rotina **RecoverMonths**, que recebe os parâmetros:

- 1) 1º parâmetro recebe um valor numérico esse valor determina quantos meses vamos retroceder, isso usando uma determinada data como base. Supondo a data 03/04/2025 e o primeiro parâmetro fosse -1, logo receberíamos o mês inicial da nossa carga 03.
- 2) 2º parâmetro recebe um valor numérico esse valor determina em que mês a carga vai parar com base em uma data determinada. Supondo a data 03/04/2025 e o segundo parâmetro 1. O mês de término da nossa carga seria 05.
- 3) 3º Parâmetro recebe um valor que vai de 1 até 3, isso define se vamos corrigir a venda (1), a compra (2) ou ambas (3)
- 4) 4º Parâmetro recebe a variável vYesterday essa variável é a data que a sub-rotina usará como parâmetro para saber qual o mês inicial e final da carga.

Sub RecoverMonths (01/01)

Seção RecorverMonths: Nela iremos percorrer o período de meses definidos, e ter controle de quais tabelas iremos carregar:

1) Loop que vai percorrer o período de meses definidos (pStartMonth,pEndMonth), recebe os parâmetros conforme a **P.5.Carga Completa (02/02)**.

2) vDate – Recebe uma data referente ao mês que o loop está percorrendo.

Exemplo: se pDateRef for igual a 03/04/2025 e o valor do vIndexMonth = -1.

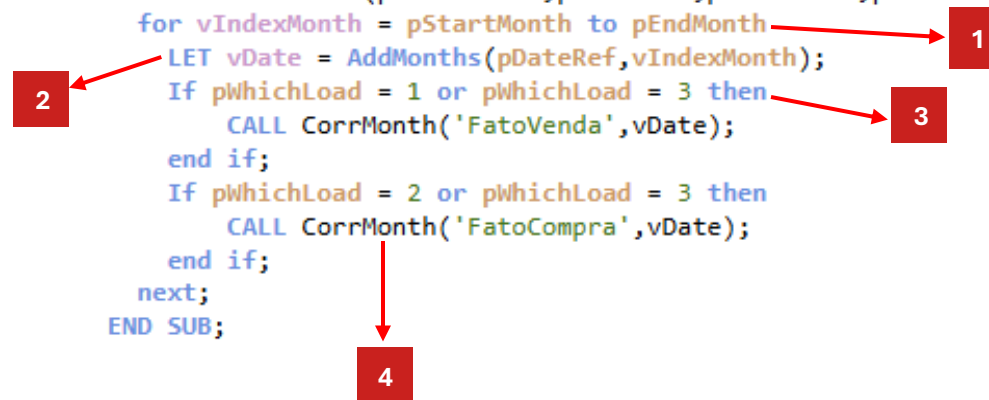
VDate será igual à 03/03/2025.

3) Estrutura condicional que faz o controle de qual tabela irá carregar. recebe os parâmetros conforme a **P.5.Carga Completa (02/02)**.

4) Sub-rotina CorrMonth(Correção Mês). Ela dará início a correção do mês, corrigindo uma tabela por vez. Recebe 2 parâmetros essenciais para as suas funcionalidades.

- O Primeiro parâmetro recebe o nome da tabela a ser carregada
- O Segundo parâmetro recebe a data de referência, a partir desta data que a sub CorrMonth vai manipular a carga.

```
/*  
    Essa sub é destinada ao controle da carga.  
    Opções para Saber em qual FATO Será gerada a carga e em qual mês  
*/  
SUB RecoverMonths(pStartMonth,pEndMonth,pWhichLoad,pDateRef)  
    for vIndexMonth = pStartMonth to pEndMonth  
        LET vDate = AddMonths(pDateRef,vIndexMonth);  
        If pWhichLoad = 1 or pWhichLoad = 3 then  
            CALL CorrMonth('FatoVenda',vDate);  
        end if;  
        If pWhichLoad = 2 or pWhichLoad = 3 then  
            CALL CorrMonth('FatoCompra',vDate);  
        end if;  
    next;  
END SUB;
```



Seção CorrMonth: Nela iremos validar se há divergências no número de linhas entre o select e o arquivo CSV/QVD existente no período mês fechado, se existir divergências, chamamos a sub-rotina CorrPeriod:

1) Variáveis de configuração para regência de data, onde a variável principal é a vDateCorrMonth, a partir dela, extraímos o primeiro dia do mês, e o último.

2) Sub-Rotinas responsáveis por buscar a quantidade de linhas do SELECT e do arquivo CSV/QVD .

3) Variável que recebe o valor total de divergência das quantidades de linhas.

4) Estrutura condicional que valida se o total de divergência é diferente de zero. Só damos prosseguimento na correção se houver divergências, se não, a sub-rotina é finalizada.

5) Sub-rotina CorrPeriod(Correção do período de um mês). Ela dará início a correção dos períodos de um mês, após localizar divergências no mês, vamos validar em qual período do mês está a divergência. Recebe 4 parâmetros essenciais para as suas funcionalidades.

```
//Essa sub vai verificar se contem divergencias no mês - entre as informações do (DW e ARQUIVOS CSV)
```

```
SUB CorrMonth(pSubFato,pDateRef)
```

```
TRACE CORRIGINDO O ARQUIVO >>> $(pSubFato);
```

```
LET vContSelect = 0;
```

```
LET vCountLinesCsv = 0;
```

```
LET vDateCorrMonth = pDateRef;
```

```
LET vFirstDayOneMonthAgo = MonthStart(vDateCorrMonth);
```

```
LET vLastDayOneMonthAgo = MonthEnd(vDateCorrMonth);
```

```
TRACE # $(vDateCorrMonth) #;
```

```
CALL $(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);
```

```
CALL $(pSubFato)CountLinesCsv('TempTableCsv',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);
```

```
LET vTotalDivergence = (vContSelect - vCountLinesCsv);
```

```
Trace $(vFirstDayOneMonthAgo) <> $(vLastDayOneMonthAgo) Total Divergencias = $(vTotalDivergence) ($(vContSelect) - $(vCountLinesCsv)) ;
```

```
If vTotalDivergence <> 0 then
```

```
CALL CorrPeriod(pSubFato,vFirstDayOneMonthAgo,vLastDayOneMonthAgo,vTotalDivergence)
```

```
End If;
```

```
END SUB;
```

1

2

3

4

5

Seção CorrMonth: Variáveis de configuração para regência de data

- 1) Variável responsável por armazenar a quantidade de linhas do resultante do Select, recebe o valor Zero como padrão
- 2) Variável responsável por armazenar a quantidade de linhas do resultante do arquivo CSV/QVD, recebe o valor Zero como padrão
- 3) Variável que recebe a data do parâmetro **pDateRef** conforme a **P.6.RecoverMonths (01/01)**.
- 4) Recebe o primeiro dia do mês, tendo como referência o parâmetro **pDateRef**.
- 5) Recebe o último dia do mês, tendo como referência o parâmetro **pDateRef**.
- 6) Variável que recebe a diferença do número de linhas entre Select e o arquivo CSV/QVD. É através dela que decidimos fazer a correção ou não.

```
1 LET vContSelect = 0;  
2 LET vCountLinesCsv = 0;  
3 Let vDateCorrMonth = pDateRef;  
4 LET vFirstDayOneMonthAgo = MonthStart(vDateCorrMonth);  
5 LET vLastDayOneMonthAgo = MonthEnd(vDateCorrMonth);  
6 LET vTotalDivergence = (vContSelect - vCountLinesCsv);
```



```
CALL $(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);
```

1

2

3

4

```
CALL $(pSubFato)CountLinesCsv('TempTableCsv',vFirstDayOneMonthAgo,vLastDayOneMonthAgo);
```

1

2

3

4

Seção CorrMonth: Nela encontramos as sub-rotinas responsáveis por buscar a quantidade de linhas do select e do arquivo CSV/QVD:

- 1) **\$(pSubFato)** - É o nome da tabela que vai ser corrigida, foi passada como parâmetro na Sub RecoverMonths('FatoVenda' ou 'FatoCompra') conforme a **P.6.RecoverMonths (01/01)**.
- 2) Esse parâmetro representa o nome da tabela temporária, que sera atribuido dentro da subrotina.
- 3) Esse parâmetro representa o primeiro dia do mês conforme a **P.8.Sub CorrMonth (02/04)**. Será o período que a sub vai contar as linhas
- 4)Esse parâmetro representa o último dia do mês conforme a **P.8.Sub CorrMonth (02/04)**. Será o período que a sub vai contar as linhas

```
If vTotalDivergence <> 0 then  
    CALL CorrPeriod(pSubFato,vFirstDayOneMonthAgo,vLastDayOneMonthAgo,vTotalDivergence)  
End If;
```

1

2

Seção CorrMonth: Este IF verifica se há divergências no mês analisado. Se houver, a sub-rotina CorrPeriod é chamada para percorrer os períodos do mês e identificar quais possuem inconsistências para correção.

- 1) Estrutura condicional que valida se a variável **vTotalDivergence** possui um valor diferente de zero.
- 2) Chama a sub-rotina CorrPeriod (Correção do Período)

```
CALL CorrPeriod(pSubFato,vFirstDayOneMonthAgo,vLastDayOneMonthAgo,vTotalDivergence)
```

1

2

3

4

Seção CorrMonth: A Sub CorrPeriod analisa em qual período do mês a divergência se encontra, dividindo o mês em períodos de 7 dias.

- 1) É o nome da tabela que vai ser corrigida conforme a **P.6.RecorverMonths (01/01)**.
- 2) Esse parâmetro representa o primeiro dia do mês conforme a **P.8.Sub CorrMonth (02/04)**.
- 3) Esse parâmetro representa o último dia do mês conforme a **P.8.Sub CorrMonth (02/04)**.
- 4) Esse parâmetro representa o total de divergência no mês.

CountLinesSelect (01/01)

//Sub que conta linhas retornadas do select Tabela "FATO_VENDA"

SUB FatoVendaCountLinesSelect(pTableName,pFirstDayOneMonthAgo,pLastDayOneMonthAgo)

1 → **\$(pTableName):**

SQL **SELECT**

COUNT(*) AS CountLineSelect → **2**

FROM CONSINCODW.FATO_VENDA FV

WHERE

FV.DTAOPERACAO BETWEEN TO_DATE('\$(pFirstDayOneMonthAgo)', 'DD/MM/YYYY') AND TO_DATE('\$(pLastDayOneMonthAgo)', 'DD/MM/YYYY');

3 → **vContSelect** = Peek('COUNTLINESELECT', 0, '\$(pTableName)');

Drop **TABLE** **\$(pTableName);** → **4**

END SUB;

Seção CountLinesSelect: Nela contamos a quantidade de linhas do select no período passado como parâmetro, armazenamos a quantidade em uma tabela temporária e depois alocamos em uma variável

- 1) Nome da tabela temporária, recebe o nome como parâmetro conforme a **P.9.Sub CorrMonth (03/04)**
- 2) Select feito para buscar a quantidade de linhas, no período passado como parâmetro.
- 3) Variável que recebe o total de linhas que está armazenado na tabela temporária.
- 4) Após a quantidade de linhas ser armazenado na variável, excluimos a tabela temporária.

```
//Sub que conta linhas retornadas do arquivo CSV da Tabela "CONSINCO_VENDAS"
```

```
SUB FatoVendaCountLinesCsv(pTableName,pFirstDayOneMonthAgo,pLastDayOneMonthAgo)
```

```
LET vYear = Year(pFirstDayOneMonthAgo);
```

```
LET vMonth = Num(Month(pFirstDayOneMonthAgo),00);
```

```
If FileSize(['$(vExtractedPath)/Consinco_Vendas_$(vYear)_$(vMonth).csv']) > 0 then
```

```
$(pTableName):
```

```
LOAD
```

```
RowNo() as Lines
```

```
FROM
```

```
[$(vExtractedPath)/Consinco_Vendas_$(vYear)_$(vMonth).csv]
```

```
(txt, utf8, embedded labels, delimiter is '|')
```

```
Where DATE(DTAOPERACAO,'DD/MM/YYYY') >= '$(pFirstDayOneMonthAgo)' and DATE(DTAOPERACAO,'DD/MM/YYYY') <= '$(pLastDayOneMonthAgo)';
```

```
vCountLinesCsv = NoOfRows('$(pTableName)');
```

```
Drop TABLE $(pTableName);
```

```
End if;
```

```
END SUB;
```

Seção CountLinesSelect: Nela contamos a quantidade de linhas do select no período passado como parâmetro, armazenamos a quantidade em uma tabela temporária e depois alocamos em uma variável

- 1) Recebe o ano e o mês referente as datas passadas como parâmetro. Essas variáveis são utilizadas para pesquisar o arquivo CSV
- 2) Estrutura que valida se o arquivo CSV/QVD existe antes de contar as linhas.
- 3) Nome da tabela temporária, recebe o nome como parâmetro conforme a **P.9.Sub CorrMonth (03/04)**.
- 4) Estrutura para contar a quantidade de linhas do arquivo CSV/QVD conforme o período passado
- 5) Variável que recebe o total de linhas que está armazenado na tabela temporária.
- 6) Após a quantidade de linhas ser armazenado na variável, excluimos a tabela temporária.

Seção CorrMonth: Nela iremos validar se há divergências no número de linhas entre o select e o arquivo CSV/QVD existente no período mês fechado, se existir divergências, chamamos a sub CorrPeriod:

- 1) Definição de variáveis para controle do loop
- 2) Estrutura de repetição que percorre o período de 7 dias passado. Começando do último dia até o primeiro. (decremento)
- 3) Variáveis para receber o último dia do período e o primeiro dia do período.
- 4) Estrutura condicional garante que o primeiro dia nunca seja maior que o último dia do período
- 5) Chamada das sub-rotinas para contagem de linhas (Select e CSV/QVD) conforme a P.11.sub CountLinesSelect (01/01) e P.12.sub CountLinesCsv (01/01).
- 6) Cálculo da diferença de linhas entre Select e arquivo
- 7) Variável recebe a atualização do valor de divergência a cada loop. O valor da divergência sempre é atualizado a após as correções até que chegue no valor 0
- 8) Estrutura condicional que valida se o total de divergência é diferente de zero. Se for, chamamos a sub de correção dia CorrDay()
- 9) Sub CorrDay(Correção Dia), corrige os dias divergentes do período
- 10) Quando a variável que armazena o total de divergências (vCtrlDivergencePeriod) atingir zero, indicando que todas as inconsistências foram corrigidas, o loop é interrompido.

```
//Essa sub vai verificar se contem divergencias nos períodos do Mês: a cada (7 dias) Começando do ultimo periodo do mês
Sub CorrPeriod(pSubFato,pFirstDayOneMonthAgo,pLastDayOneMonthAgo,pCtrlDivergence)
  LET vStep = -7;
  SET fCalcDvgCtrl = ($1 - ($2 - $3));
  LET vCtrlDivergencePeriod = pCtrlDivergence;
  for vIndexPeriod = pLastDayOneMonthAgo to pFirstDayOneMonthAgo step vStep
    LET vLastDayPeriod = date(vIndexPeriod);
    LET vFirstDayPeriod = date(vIndexPeriod+(vStep+1));
    If day(vFirstDayPeriod) > day(vLastDayPeriod) then
      vFirstDayPeriod = MonthStart(vLastDayPeriod);
    end if;
    CALL $(pSubFato)CountLinesSelect('TempCountSelect',vFirstDayPeriod,vLastDayPeriod);
    CALL $(pSubFato)CountLinesCsv('TempTableCsv',vFirstDayPeriod,vLastDayPeriod);
    LET vDivergencePeriod = (vContSelect - vCountLinesCsv);
    vCtrlDivergencePeriod = $(fCalcDvgCtrl(vCtrlDivergencePeriod,vContSelect,vCountLinesCsv));
    If vDivergencePeriod <> 0 then
      CALL CorrDay(pSubFato,vFirstDayPeriod,vLastDayPeriod,vDivergencePeriod);
    End If;
    If vCtrlDivergencePeriod = 0 then
      Exit for;
    end if;
  next;
END SUB;
```

Seção CorrPeriod: Variáveis de manipulação

- 1) Variável responsável por armazenar de quanto em quanto o loop vai percorrer, nesse caso o loop vai percorrer do primeiro dia do mês até o último de 7 em 7
- 2) Esta variável recebe uma função que atualiza o valor da divergência a cada loop em que uma inconsistência é identificada e corrigida. Seu objetivo é reduzir gradualmente as divergências até que o valor seja zerado
- 3) Recebe o total de divergência do mês conforme a **P.14.Sub CorrMonth (04/04)**
- 4) Recebe o último dia do período percorrido.
- 5) Recebe último dia do período percorrido – 7, ou seja. Se o último dia for igual a 31, FirstDayPeriod vai receber 24. Formando o primeiro período a ser verificado do dia 31 até 24
- 6) Variável que recebe a diferença do número de linhas entre Select e o arquivo CSV/QVD. Conforme o período passado.

```
1 LET vStep = -7;  
2 SET fCalcDivgCtrl = ($1 - ( $2 - $3));  
3 LET vCtrlDivergencePeriod = pCtrlDivergence;  
4 LET vLastDayPeriod = date(vIndexPeriod);  
5 LET vFirstDayPeriod = date(vIndexPeriod+(vStep+1));  
6 LET vDivergencePeriod = (vContSelect - vCountLinesCsv);
```

```
1 ← If vDivergencePeriod <> 0 then  
    CALL CorrDay(pSubFato,vFirstDayPeriod,vLastDayPeriod,vDivergencePeriod);  
End If;
```

2

Seção CorrPeriod: Este IF verifica se há divergências no período analisado. Se houver, a sub-rotina CorrDay é chamada para percorrer os dias do Período e identificar quais possuem inconsistências para correção.

- 1) Estrutura condicional que valida se a variável **vDivergencePeriod** possui um valor diferente de zero.
- 2) Chama a sub-rotina CorrDay(Correção Dia)

```
CALL CorrDay(pSubFato,vFirstDayPeriod,vLastDayPeriod,vDivergencePeriod);
```

1

2

3

4

Seção CorrPeriod: A Sub CorrPeriod analisa em qual período do mês a divergência se encontra, dividindo o mês em períodos de 7 dias. E chama a sub-rotina CorrDay

- 1) É o nome da tabela que vai ser corrigida, conforme a **P.6.RecoverMonths (01/01)**.
- 2) Esse parâmetro representa o primeiro dia do período conforme a **P.13.Sub CorrPeriod (01/03)**.
- 3) Esse parâmetro representa o último dia do período conforme a **P.13.Sub CorrPeriod (01/03)**.
- 4) Esse parâmetro representa o total de divergência do período.

Seção CorrDay: Nela iremos validar se há divergências nos dias de um período passado. Caso encontre alguma divergência, chama a sub-rotina de correção de carga CALL \$(pSubFato)(vDateRef):

- 1) Recebe o total de divergência do período conforme a P.14.sub CorrPeriod (03/03)
- 2) Estrutura de repetição que percorre o período de 7 dias passado. Começando do último dia até o primeiro. (decremento)
- 3) Chama as sub-rotinas para realizar a contagem de linhas passando o dia como parâmetro conforme a P.11.sub CountLinesSelect (01/01) e P.12.sub CountLinesCsv (01/01) .
- 4) Recebe o total de divergência do dia percorrido
- 5) Variável recebe a atualização do valor de divergência a cada lopp. O valor da divergência sempre é atualizado a após as correções até que chegue no valor 0
- 6) Estrutura condicional que valida se o total de divergência é diferente de zero. Se for, chamamos a sub de correção CALL \$(pSubFato)(vDateRef) para realizar a correção do dia.
- 7) Quando a variável que armazena o total de divergências (vCtrlDivergenceDay) atingir zero, indicando que todas as inconsistências foram corrigidas, o loop é interrompido.

```

/*Essa sub Verifica se contem divergências no dia a dia dos períodos
. Após localizar os dias divergente ela chama a correção do arquivo*/
SUB CorrDay(pSubFato,pFirstDayPeriod,pLastDayPeriod,pCtrlDivergencePeriod)
1  LET vCtrlDivergenceDay = pCtrlDivergencePeriod;

    for vDateRef = pLastDayPeriod to pFirstDayPeriod step -1 2
        vDateRef = Date(vDateRef)

3  CALL $(pSubFato)CountLinesSelect('TempCountSelect',vDateRef,vDateRef);
   CALL $(pSubFato)CountLinesCsv('TempTableCsv',vDateRef,vDateRef);

        LET vDivergenceDay = (vContSelect - vCountLinesCsv); 4

5  vCtrlDivergenceDay = $(fCalcDivgCtrl(vCtrlDivergenceDay,vContSelect,vCountLinesCsv));

        //Comparando os totais de linhas do arquivoQVD e do resultado da consulta no DW
6  If vDivergenceDay <> 0 then
            Trace Correction;
            CALL $(pSubFato)(vDateRef);
        end if;

7  If vCtrlDivergenceDay = 0;
        Exit For;
    End If;

    next;

End SUB;

```



```
CALL $(pSubFato)(vDateRef);
```

1

2

Seção CorrDay: Sub \$(pSubFato)(vDateRef) é chamada para corrigir o arquivo no dia que está a divergência utilizando a data vDateRef como parâmetro.

- 1) É o nome da tabela que vai ser corrigida, conforme a **P.6.RecoverMonths (01/01)**.
- 2) É a variável que está percorrendo os dias do período



www.pegpese.com.br
bomtever.pegpese.com.br



www.linkedin.com/company/pegpesehortifruti



@pegpese



11 97252-6182